

# MedLedger AI: Patient-Controlled Health Records with Encrypted Storage, an Ethereum-Anchored Record History and Privacy-Preserving AI Analysis

Pankaj Baduwal  
Department of Information  
Technology  
KPR Institute of Engineering  
and Technology  
Coimbatore, India  
23IT040@kpriet.ac.in

Yamraj Chaudhary  
Department of Information  
Technology  
KPR Institute of Engineering  
and Technology  
Coimbatore, India  
23IT069@kpriet.ac.in

Vikasini S  
Department of Information  
Technology  
KPR Institute of Engineering  
and Technology  
Coimbatore, India  
23IT064@kpriet.ac.in

Dr. M. Ashok Kumar  
Associate Professor,  
Department of Information  
Technology  
KPR Institute of Engineering  
and Technology  
Coimbatore, India

**Abstract**—Medical records are usually split across hospitals, clinics and laboratories, patients cannot see who opens them, and the reports themselves are hard for non-specialists to read. This paper presents MedLedger AI, a web platform that combines encrypted storage, patient-controlled sharing, a tamper-evident record history anchored on Ethereum and an AI engine that explains reports in plain language. Every document is encrypted with its own AES-256-GCM key bound to the record and the patient, fingerprinted with SHA-256 and verified again each time it is opened. A single per-patient permission list decides who can see a record, and each upload and sharing decision is appended to a hash-linked history and written to a HealthRecords smart contract, either by a server relayer or from the patient’s MetaMask wallet. The AI engine extracts text with OCR, removes personal details before analysis and reports vital signs, abnormal values, conditions and drug interactions. Six role-based workspaces are provided in English, Nepali and Hindi. In the evaluation, all 5,000 tampered records and all 1,200 tampered history entries were rejected, an 81-cell access-control matrix matched the intended policy exactly, on 2,000 held-out synthetic reports de-identification removed every personal detail even without the registered patient name and abnormal-value detection made no error, value recall on phone photographs of reports was 98%, and relayer writes cost 26.7k–48.7k gas, or about 1k gas per record when records are anchored in batches of 50. The system passes 285 automated tests. Validation on real reports and with users is under way; this paper reports the synthetic evaluation and the study protocol.

**Keywords**—Electronic health records, blockchain, Ethereum smart contract, AES-256-GCM, SHA-256, patient consent, role-based access control, de-identification, clinical AI, OCR.

## I. INTRODUCTION

Digital health information is produced by many organisations at once: hospitals keep admission notes and prescriptions, diagnostic laboratories issue test reports, and insurers hold claim documents. Because each organisation stores its own copy in its own system, a patient’s history is fragmented, and the patient has little control over, or visibility into, who reads it [1], [2]. Centralised stores are also attractive targets; recent systematic reviews list breaches, weak interoperability, inconsistent consent and lack of patient control as the main open problems of electronic health record (EHR) systems [3], [4].

Blockchain technology has been proposed to address integrity and consent, because an append-only ledger can prove that a record existed in a given form and that a permission was granted at a given time [1], [2]. Blockchain alone, however, does not make records understandable. Laboratory reports are

dense with abbreviations and reference ranges that most patients cannot interpret, and clinicians spend time reading long histories. Large language models (LLMs) now summarise clinical text at a level comparable to experts [5], [6], but sending identifiable records to an external model conflicts with the privacy goals that motivate the blockchain layer in the first place.

A second gap is evaluation. Many blockchain EHR prototypes report gas cost or transaction latency but do not test whether tampering is actually detected, whether the access policy is enforced for every role, or how accurate the AI output is. This paper describes MedLedger AI, a working system that brings these requirements together in one patient-centric platform, and evaluates it along all of these dimensions. Its contributions are:

- 1) an encrypted record vault in which every file has its own AES-256-GCM key, bound by associated data to its record and patient, and a SHA-256 fingerprint re-verified on every download;
- 2) a single consent model (per patient–doctor permission) from which every listing, download and count is derived, recorded in a hash-linked history and on an Ethereum contract through either a server relayer or the patient’s own wallet;
- 3) a privacy-preserving AI pipeline (OCR, de-identification with an independent re-check, a built-in clinical rule engine and an optional LLM) that explains reports in plain language in three languages;
- 4) a reproducible evaluation: 6,200 tamper trials, an 81-cell access-control matrix, AI accuracy on 2,000 held-out synthetic reports (half deliberately hard) and 180 scanned and photographed pages, cryptographic cost, concurrency up to 100 clients, gas, usability and 285 automated tests, together with a comparison against recent systems.

Section II reviews related work, Sections III–V present the architecture, methods and implementation, Section VI describes the evaluation setup, Section VII reports and discusses results and Section VIII concludes.

## II. RELATED WORK

Early work showed how a blockchain can manage medical permissions. MedRec [7] stores pointers and access permissions on Ethereum while the data stays with providers, and Ancile [8] adds privacy-preserving access control and interoperability using smart contracts. Scoping and systematic reviews [2]–[4] confirm integrity, auditability and consent as the main motivations, and note that most prototypes stop at storage and access control.

Recent systems combine blockchain with encryption and off-chain storage. HealthChain [9] integrates encryption, consent management and interoperability and reports faster access and fewer breaches than baseline designs. Haddad *et al.* [10] build a patient-centric Ethereum EHR with two-layer (AES and ECC) encryption and IPFS storage. Tahir *et al.* [11] present a modular management framework, and Singh *et al.* [12] combine Ethereum, IPFS, AES-256 and role-based access control with lower gas fees than earlier work. Dutta and Barman [13] use smart contracts for storing and sharing EHRs, Katoon and Turukmane [14] synchronise records between Hyperledger Fabric and Ethereum, Pu *et al.* [15] add risk-aware smart-contract access control, and Ullah *et al.* [16] use fine-grained attribute-based encryption with decentralised storage. Closest to our consent design, Romel *et al.* [17] separate AES-256-GCM encrypted off-chain storage from one permission contract per patient with EIP-712 signatures.

On the AI side, adapted LLMs can outperform medical experts in clinical text summarisation [5], LLMs encode substantial clinical knowledge [6], and reviews [18] and evaluation studies [19] stress validation of generated summaries. Automated de-identification of free text [20] and OCR [21] are established building blocks for processing scanned reports.

Table I summarises the gap. Existing blockchain EHR systems concentrate on encryption, storage and access control; none of those reviewed combines them with AI explanation of the reports, OCR of scanned documents and a multilingual, plain-language interface, and few verify integrity again when a file is read. MedLedger AI targets this combination while keeping the AI step on de-identified text. A quantitative comparison follows in Section VII-F.

TABLE I  
FEATURE COMPARISON WITH RELATED BLOCKCHAIN EHR SYSTEMS (– = NOT REPORTED)

| Work            | Encryption | Chain             | Patient consent | Check on read | AI | OCR | Multilingual |
|-----------------|------------|-------------------|-----------------|---------------|----|-----|--------------|
| MedRec [7]      | –          | Ethereum          | ✓               | –             | –  | –   | –            |
| HealthChain [9] | ✓          | ✓                 | ✓               | –             | –  | –   | –            |
| Haddad [10]     | AES + ECC  | Ethereum          | ✓               | –             | –  | –   | –            |
| Singh [12]      | AES-256    | Ethereum          | RBAC            | Hash          | –  | –   | –            |
| Katoon [14]     | AES-256    | Fabric + Ethereum | –               | –             | –  | –   | –            |
| Ullah [16]      | ABE        | Ethereum          | Attributes      | –             | –  | –   | –            |
| Romel [17]      | AES-GCM    | Ethereum          | EIP-712         | GCM tag       | –  | –   | –            |
| MedLedger AI    | AES-GCM    | Ethereum          | ✓ + wallet      | Tag + hash    | ✓  | ✓   | EN/NE/HI     |

### III. SYSTEM ARCHITECTURE

MedLedger AI is organised in four layers (Fig. 1). The presentation layer is a React and TypeScript single-page application with six role-based workspaces (patient, doctor, hospital, laboratory, insurance and administrator), a language switcher (English, Nepali, Hindi) and an optional MetaMask connection. The application layer is a Node.js/Express REST API written in TypeScript. It contains five services: authentication (JWT sessions, bcrypt, lockout and Sign-In with Ethereum), role and consent middleware, the record vault, the AI engine and the record history with its relay queue.

The data layer holds an encrypted state store, in which lists containing personal data (readings, prescriptions, admissions, claims, permissions) are sealed individually with AES-256-GCM, and the encrypted files themselves, stored in MongoDB GridFS when a database is configured and in a local encrypted store otherwise. A master key held outside the data store wraps every file key. The blockchain layer is the HealthRecords contract (Solidity 0.8.20), deployed on the Sepolia test network or on a local Hardhat network. Only fingerprints and pseudonymous addresses are written on-chain; document contents and personal identifiers never leave the application.

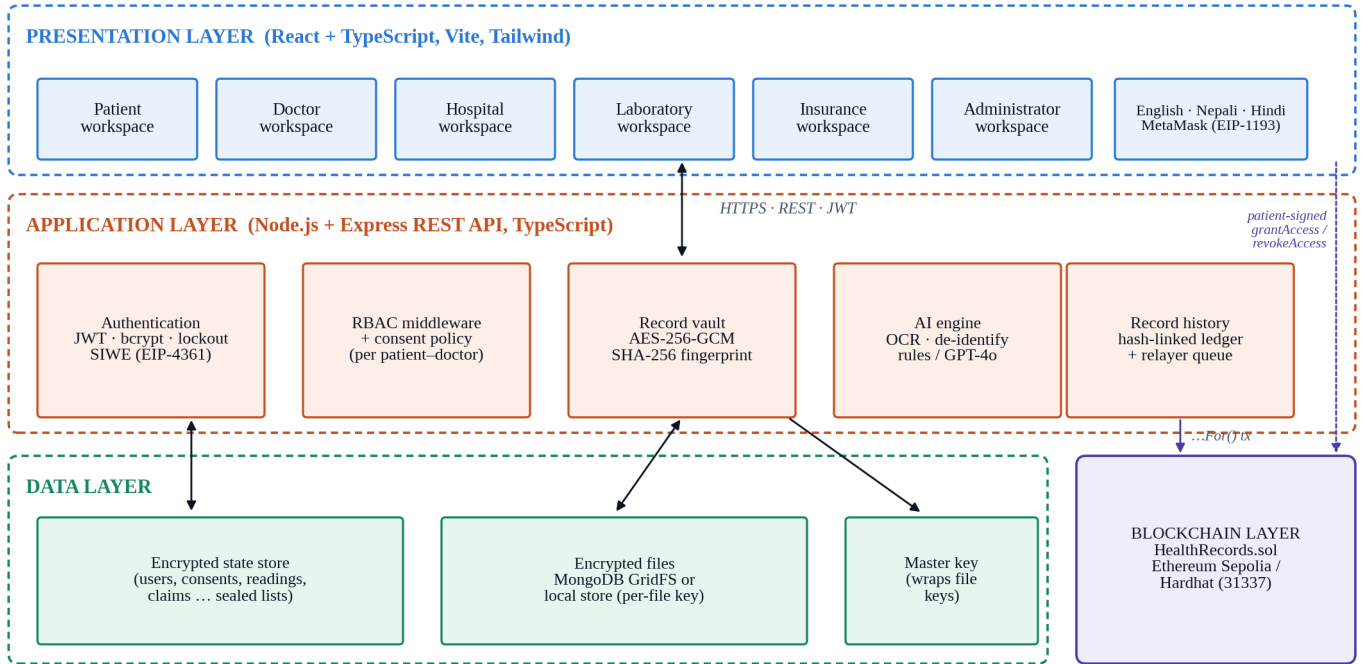


Fig. 1. Four-layer architecture of MedLedger AI.

#### IV. METHODOLOGY

##### A. Encrypted Record Vault

Fig. 2 shows the upload and download paths. When a file  $M$  is uploaded, the server checks its type and size and computes the fingerprint

$$h = \text{SHA-256}(M), \quad (1)$$

using the Secure Hash Standard [24]. A fresh 256-bit data key  $K$  and a 96-bit initialisation vector  $IV$  are drawn from a cryptographically secure generator, and the file is encrypted with the Galois/Counter Mode [23]:

$$(C, T) = \text{AES-256-GCM}(K, IV, M, A), \quad (2)$$

where  $T$  is the authentication tag and the associated data  $A = (\text{record id}, \text{patient id})$  binds the ciphertext to its record and owner, so a file copied into another record or moved to another patient fails authentication.  $K$  is then wrapped with the master key using the same mode, with the record id as associated data, so a key cannot be transplanted between records and rotating the master key requires re-wrapping keys only, not re-encrypting files. The ciphertext, its SHA-256 checksum and the metadata ( $h$ , wrapped key,  $IV$ ,  $T$ ) are stored, a history entry is appended (Section IV-C) and text is extracted for the AI engine.

On every download the server first checks the caller's role and permission. It then checks the stored checksum, unwraps  $K$ , verifies  $T$ , decrypts to  $M'$  and releases the file only if  $\text{SHA-256}(M') = h$ . A failed check refuses the download with HTTP 409 and writes a tamper event to the audit log, so a modified file is never delivered silently. The four checks are deliberately redundant; Section VII-B measures which of them catches each kind of tampering.

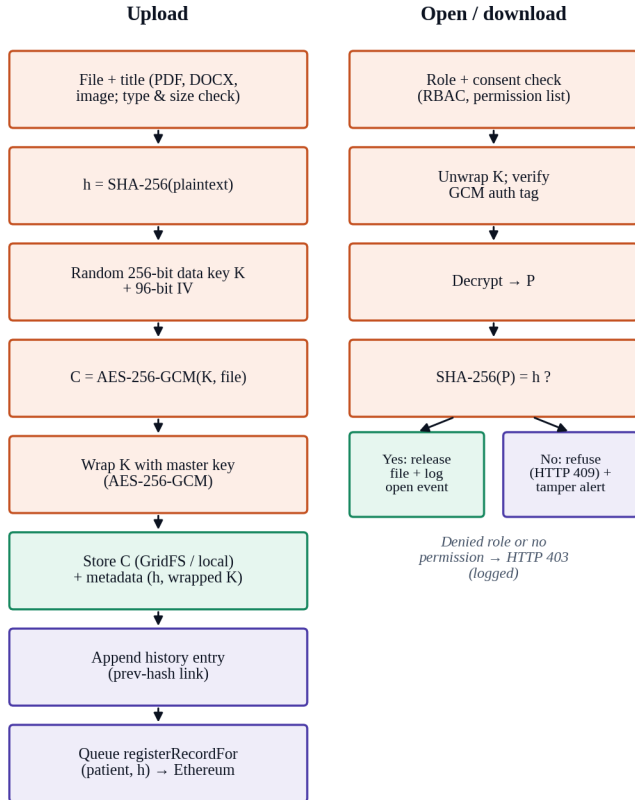


Fig. 2. Upload (left) and open/download (right) paths of the encrypted record vault.

##### B. Patient-Controlled Consent

Access follows role-based access control [25] refined by an explicit consent list. For each patient–doctor pair the list holds

one entry whose state is *pending*, *granted*, *declined* or *revoked*. A doctor may ask for access with an optional reason; only the patient can grant, decline or revoke it, apart from an audited administrator override. Every record listing, download, dashboard count and notification is computed from this one list, so different pages cannot disagree. Record lists are further scoped by role: doctors see only patients who granted them access, laboratories see their own uploads and insurers see only documents attached to a claim.

Fig. 3 shows the two ways a decision reaches the blockchain. In the default path the patient decides inside the application; the API updates the list, appends a history entry and queues a relay transaction. In the wallet path the patient links a MetaMask account by signing a Sign-In with Ethereum message [27] and confirms the change either with an EIP-712 typed-data signature [26] or by sending grantAccess/revokeAccess from their own wallet. The server verifies the signature, or the transaction receipt and its event, before applying the change, and records the patient's own transaction instead of sending a second one.

##### C. Record History and Smart Contract

Every upload and every sharing decision becomes an entry  $E_i$  in a hash-linked history. With  $p_i$  the entry payload (for example patient, doctor and fingerprint),  $t_i$  its timestamp and  $\parallel$  concatenation of a canonical encoding,

$$H_i = \text{SHA-256}(i \parallel \text{type}_i \parallel t_i \parallel H_{i-1} \parallel p_i), \quad (3)$$

with  $H_{-1} = 0$ . Verifying the chain recomputes every  $H_i$  and its link; editing, deleting, inserting or reordering any entry breaks a link and is reported at the first broken entry. The history is saved with the server state and survives restarts.

The HealthRecords contract (Solidity 0.8.20) stores record fingerprints and permissions and emits RecordAdded, AccessGranted and AccessRevoked events. Patients with a wallet call registerRecord, grantAccess and revokeAccess directly. For patients without a wallet the server acts as a *relayer*: the account that deployed the contract may call registerRecordFor(patient, h), grantAccessFor(patient, doctor) and revokeAccessFor(patient, doctor), and no other account can. Patients are represented by their linked wallet address or, if they have none, by a pseudonymous address derived from their account identifier. Transactions are queued and sent one at a time in the background so that user actions never wait for block confirmation; entries not yet confirmed are re-sent after a restart, and a fingerprint already on the chain is recognised instead of causing a failed transaction. Without a configured contract the history is still kept and verified on the server. Algorithm 1 summarises the relayer.

##### Algorithm 1: Background relayer for a history entry $E$

**Input:** entry  $E$  (type, payload), contract  $C$ , relayer key  $k$

```

1: append  $E$  to the history;  $E.\text{chain} \leftarrow \text{queued}$ ; save
2: if  $E$  was sent from the patient's wallet then
3:    $E.\text{chain} \leftarrow \text{confirmed}(\text{patient tx})$ ; return
4: enqueue job( $E$ ) // jobs run one at a time
5: on job( $E$ ):
6:   if  $E$  is a record and  $C.\text{verifyRecord}(h)$  then
7:      $E.\text{chain} \leftarrow \text{confirmed}(\text{already on chain})$ ; return
8:    $\text{tx} \leftarrow \text{registerRecordFor} \mid \text{grantAccessFor} \mid \text{revokeAccessFor}$ , signed with  $k$ 
9:    $E.\text{chain} \leftarrow \text{sent}(\text{tx.hash})$ ; save
10:   $r \leftarrow \text{wait for 1 confirmation of tx}$ 
11:   $E.\text{chain} \leftarrow \text{confirmed}(r.\text{block})$  if  $r.\text{status} = 1$  else failed
12: on restart: re-enqueue queued entries; poll receipts of sent ones

```

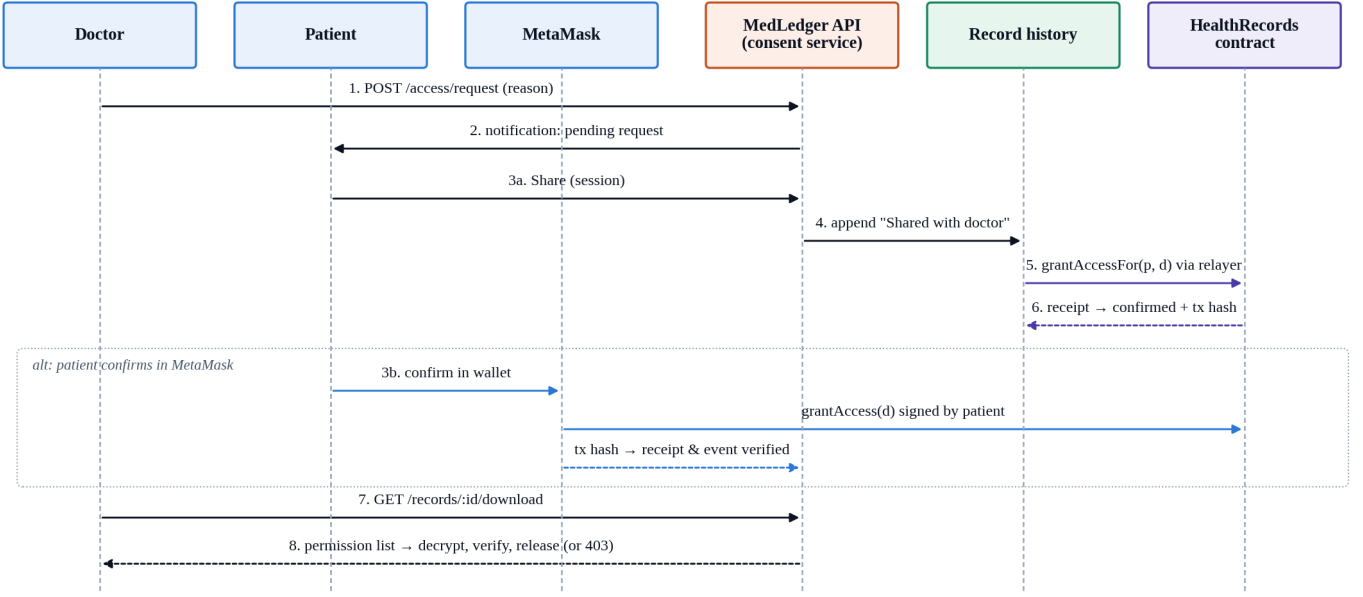


Fig. 3. Consent sequence: a doctor asks, the patient shares in the application (relay path) or confirms in MetaMask (wallet path), and the doctor opens the record.

#### D. Privacy-Preserving AI Engine

Fig. 4 shows the AI pipeline. Text comes from the PDF text layer or from Tesseract OCR [21] in a small worker pool; noisy photographs are first cleaned (median filter, division by a blurred background), numbers read with low confidence are marked “check against the report”, and pages below 80% confidence stay on the built-in engine. De-identification removes names, dates of birth, telephone numbers, e-mail and postal addresses, identity numbers, clinician and facility names [20]: labelled fields, the registered patient name, every name the document itself labels wherever it recurs, people after cue phrases and roles (“accompanied by her husband ...”), capitalised pairs containing a common Nepali or Indian name, and names in Devanagari; an independent pass confirms none remain. A clinical engine extracts 30 kinds of vital signs and lab values in four layouts, converts SI units and judges each value against the range printed on the report, else sex-specific adult ranges; it also checks 77 drugs against 41 interaction rules. With an OpenAI key, GPT-4o receives only de-identified text and its numbers are re-checked. Every result states that it is not a diagnosis.

#### E. Plain Language and Localisation

Because the target users include patients without technical background, interface text was rewritten using a fixed word list (for example “Unchanged since upload ✓” instead of “SHA-256 verified”, “Share with doctor” instead of “grant access on-chain”), and raw status codes were replaced by actionable messages. An automated crawler scores each page from 10, deducting for jargon, all-capital labels, sentences above 28 words, vague button labels and status codes. About 1,650 interface strings were translated into Nepali and Hindi; a DOM-level translator swaps text, placeholders and labels at run time while leaving names, report text and AI output untouched.

#### F. Threat Model

We assume an attacker who can read and modify the storage layer (database, file store or backups) but not the master key, a curious or compromised user account of any role, and an external AI provider that should not learn identities. The relay is trusted to submit, but not to forge, patient-signed decisions.

Table II maps each threat to its mitigation and to the experiment that tests it.

TABLE II  
THREATS, MITIGATIONS AND EVALUATION

| Threat                               | Mitigation                             | Tested in  |
|--------------------------------------|----------------------------------------|------------|
| File modified in storage             | GCM tag, SHA-256 fingerprint           | Fig. 5     |
| Key or file moved between records    | Associated data binds key and file     | Fig. 5     |
| History edited, deleted or reordered | Hash links, on-chain anchor            | Sec. VII-B |
| Read without permission              | RBAC + consent list                    | Fig. 6     |
| Password guessing                    | bcrypt, rate limit, logout             | Table V    |
| Forged or replayed consent           | EIP-712 / receipt check, tx uniqueness | Table V    |
| Identity leak to external AI         | De-identification + re-check           | Fig. 7     |
| Personal data on-chain               | Fingerprints and pseudonyms only       | Design     |

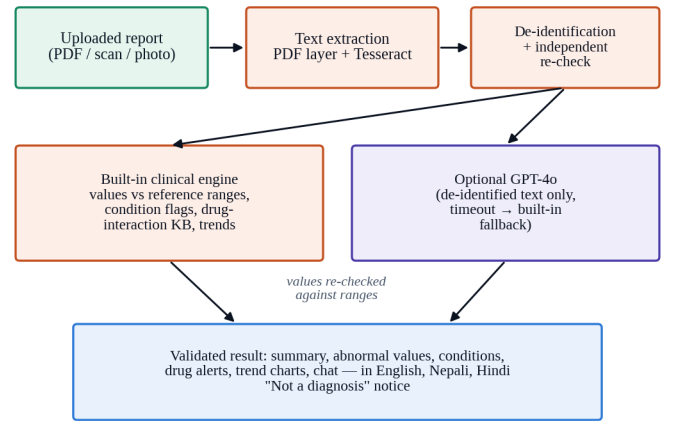


Fig. 4. Privacy-preserving AI pipeline.

#### V. IMPLEMENTATION

The client uses React 18, TypeScript, Vite, Tailwind CSS and Zustand; the server uses Node.js, Express and TypeScript with Zod validation, Helmet headers, rate limiting and Morgan logging. Files are stored in MongoDB GridFS or a local encrypted store; the contract is compiled and tested with

Hardhat and accessed with ethers.js v6. Tesseract.js performs OCR and the OpenAI API is optional. Table III lists what each role can do; administrators additionally manage accounts and see the full record history, which is withheld from all other roles because it contains patient and doctor identifiers.

The deployed system, with demonstration accounts for all six roles and sample data, is available at <LIVE-DEMO-URL>. The deployment uses the Sepolia test network, so every confirmed history entry links to its transaction on a public block explorer. Source code, evaluation scripts and raw results are kept in the project repository so that every number in Section VII can be regenerated.

TABLE III  
ROLE PERMISSIONS

| Role       | Can see                            | Can add / decide                     |
|------------|------------------------------------|--------------------------------------|
| Patient    | Own records, readings, activity    | Upload; share, decline, stop sharing |
| Doctor     | Records of patients who shared     | Request access; notes, prescriptions |
| Hospital   | Staff, admissions, prescriptions   | Admit, discharge, manage staff       |
| Laboratory | Own uploads and samples            | Upload results; check fingerprints   |
| Insurance  | Documents attached to claims       | Approve, reject, pay claims          |
| Admin      | Accounts, security, record history | Create or disable accounts           |

## VI. EVALUATION SETUP

### A. Environment

All measurements were taken on one Linux machine with Node.js 22 and local encrypted-file storage; gas was measured on a Hardhat network with the Solidity optimiser (200 runs). The server was started in its test configuration with a fresh state file for each experiment, and HTTP experiments used the public REST API exactly as the client does.

### B. Synthetic Evaluation Dataset

A seeded generator produces laboratory reports with known ground truth (Table IV): a labelled header (facility, name, age and sex, date of birth, telephone, address, optional ID and e-mail), four to nine results in one of four layouts, a narrative mentioning the patient without a label and a signature; about 60% of values are abnormal. A *hard* set adds laboratory-specific printed ranges ( $\pm 8\%$ ), values near the limits, SI units, relatives, a second clinician, a sample collector, Devanagari names and names outside the de-identifier’s list. Development used separate seeds; each held-out set was generated once afterwards. For OCR, 60 reports were rendered as clean scans, phone photographs and degraded photographs.

TABLE IV  
EVALUATION DATASET (HELD-OUT SEEDS)

| Item                                | Count           |
|-------------------------------------|-----------------|
| Synthetic reports (standard / hard) | 1,000 / 1,000   |
| Lab and vital values                | 7,461 / 7,434   |
| ... of which abnormal               | 4,463 / 4,310   |
| Personal details                    | 11,233 / 14,079 |
| ... unlabelled in the narrative     | 2,000 / 3,683   |
| Drug pairs (interacting / not)      | 178 / 178       |
| Pages (clean / photo / degraded)    | 60 / 60 / 60    |
| Tamper trials (file / history)      | 5,000 / 1,200   |

### C. Metrics

For detection tasks (abnormal values, personal details, drug interactions, tampering) with true positives TP, false positives FP and false negatives FN we report

$$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad (4)$$

$$F_1 = 2PR / (P + R), \quad (5)$$

and specificity  $TN/(TN + FP)$ . A personal detail counts as removed only if its value no longer appears anywhere in the output, so a name removed from the header but left in the narrative counts as a miss. A lab value counts as found if the extracted number matches the ground truth to within rounding. OCR quality is the character error rate

$$CER = (S + D + I) / N, \quad (6)$$

where  $S$ ,  $D$  and  $I$  are the substitutions, deletions and insertions of the Levenshtein alignment and  $N$  is the length of the reference text after whitespace normalisation. Latency is reported as the median and 95th percentile (p95).

## VII. RESULTS AND DISCUSSION

### A. Functional Verification

All tests in Table V pass on the final build and from a fresh clone of the repository. The server suite includes an end-to-end test that starts a Hardhat node, deploys the contract, points the server at it and confirms that an upload, a share and a revocation each become a confirmed transaction with the expected event and parties. The browser suite drives the full consent scenario across four roles and opens every page of all six workspaces. The table also lists the misuse cases covered by these suites.

TABLE V  
AUTOMATED TEST SUITES AND MISUSE CASES

| Suite / case                             | Tests / observed result                               |
|------------------------------------------|-------------------------------------------------------|
| Contract (Hardhat, Mocha)                | 22: registration, batches, grant/revoke, relayer, gas |
| Server (node:test)                       | 213: auth, crypto, OCR, AI, consent, history, chain   |
| Client (Vitest)                          | 27: forms, chart, wallet flow, languages              |
| Browser (Playwright)                     | 23: consent flow, all pages, phone layout             |
| Total                                    | 285, all passing                                      |
| Request without a valid token            | HTTP 401                                              |
| Repeated wrong passwords                 | Account locked for a set period                       |
| Consent signed by another wallet         | Rejected (HTTP 401)                                   |
| Same consent transaction sent twice      | Rejected (HTTP 409)                                   |
| Other account calls grantAccessFor       | Contract reverts                                      |
| Public test key used off the local chain | Contract writes disabled                              |

### B. Security Evaluation

*Tamper detection.* Each of 500 trials encrypted a random file of 2 KB to 200 KB with the production code and then applied ten kinds of tampering, from a single bit flip in the ciphertext to moving the record to another patient. To test the cryptographic checks rather than the cheap checksum, the attacker also recomputed the stored ciphertext checksum wherever the file was changed. Fig. 5 shows that all 5,000 tampered records were refused while all 500 untouched controls were released (no false alarms). The GCM tag caught every change to the ciphertext, IV or tag and every file moved between records or patients; key unwrapping caught modified or transplanted keys, because the key is bound to its record; the SHA-256 fingerprint caught edited metadata. The fingerprint check is therefore a second line behind the tag, and the associated data closes the swap attacks that a tag alone would not detect.

*Record-history tampering.* On 50-entry histories, six attacks (editing a payload field or timestamp, deleting, swapping, editing and re-hashing, and inserting a forged entry) were applied at a random position 200 times each. All 1,200 were detected and located within one entry of the change, including an attacker who edits an entry and recomputes its own hash, and one who inserts a correctly hashed forged entry; both break the

link to the next entry. All 200 untouched histories verified as intact.

**Access-control matrix.** Nine actors, including the same doctor before consent, after consent and after revocation, attempted nine actions on one patient’s record through the REST API (Fig. 6). Actions were judged by their effect, not only their status code: for example, an upload by another patient is accepted but lands in that patient’s own record (shown as 409). All 81 outcomes matched the intended policy. Revocation took effect immediately, and a revoked doctor could no longer list, open or verify the record. The matrix also exposed a defect: the server accepted uploads from insurance accounts into any patient’s record, although the client never offered this. The rule was tightened, and a regression test was added.

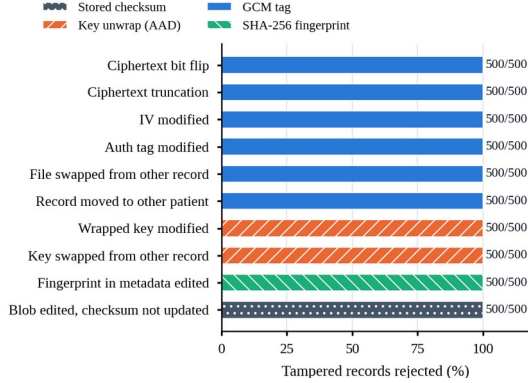


Fig. 5. Tampered records rejected, coloured by the check that caught them (500 trials per attack; 500 untouched controls were all released).

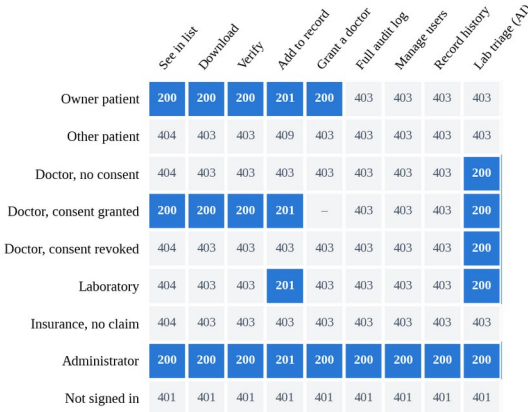


Fig. 6. Access-control matrix measured through the REST API. Blue = allowed, grey = denied; numbers are HTTP status codes (409 = accepted but stored in the caller’s own record). All 81 cells match the intended policy.

### C. AI Evaluation

**De-identification.** Fig. 7 reports recall when the registered patient name is *not* supplied. Labels and patterns alone removed 81.2% of details in the standard set and 73.6% in the hard set, missing unlabelled mentions of the patient and most relatives. Re-using names the document labels, cue phrases, roles and the name list raised recall to 100% on both sets (25,312 details, including 1,137 names outside the list and 427 in Devanagari) at under 1 ms per report. On 20 hand-written sentences with phrasings the generator never uses, 81.6% of name words were removed (15.8% before), so the registered name should still always be passed.

**Extraction and abnormal-value detection.** On both held-out sets every value was found without spurious values and every status was correct (Table VI). The 28 false alarms of an earlier

version ( $F_1$  0.997) all came from ground truth that used sex-neutral ranges where the engine used the patient’s sex, so the truth was corrected, not the engine. On the hard set that version found only 89.9% of values ( $F_1$  0.893) because it read neither SI units nor the laboratory’s own ranges. Results were identical on de-identified text, and the drug checker separated all 178 interacting pairs from 178 non-interacting ones.

TABLE VI  
AI ENGINE RESULTS ON HELD-OUT REPORTS (STANDARD / HARD)

| Task                                | Metric              | Result           |
|-------------------------------------|---------------------|------------------|
| De-id., labels and patterns         | Recall              | 81.2 / 73.6%     |
| De-id., full (no registered name)   | Recall              | 100 / 100%       |
| De-id., unseen phrasings (38 names) | Recall              | 81.6%            |
| Value extraction                    | Recall / precision  | 100 / 100%       |
| Status (low / normal / high)        | Accuracy            | 100 / 100%       |
| Abnormal values                     | $F_1$ / specificity | 100 / 100%       |
| Same, after de-identification       | $F_1$               | Unchanged        |
| Drug interactions (356 pairs)       | P / R / $F_1$       | 100 / 100 / 100% |

**Scans and photographs.** Table VII and Fig. 8: value recall was 98.6% on clean scans, 98.1% on phone photographs and 86.9% on degraded photographs (45.4% before the noise-driven clean-up, which with the worker pool also cut 19 s to 2.5 s per page). On degraded photographs 6% of extracted values were misread, but all except 0.5% carried the “check against the report” mark. With the registered name, de-identification removed every detail from all pages (99.7% without it).

TABLE VII  
PIPELINE ON SCANS AND PHOTOGRAPHS (60 PAGES EACH, %)

| Input          | CER | Value recall | Abn. $F_1$ | De-id recall | s / page |
|----------------|-----|--------------|------------|--------------|----------|
| Digital text   | 0   | 100          | 100        | 100          | —        |
| Clean scan     | 0.3 | 98.6         | 99.4       | 100.0        | 1.2      |
| Phone photo    | 0.5 | 98.1         | 98.7       | 100.0        | 1.1      |
| Degraded photo | 1.7 | 86.9         | 93.1       | 100.0        | 2.5      |

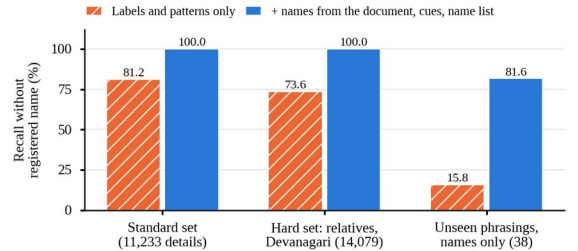


Fig. 7. De-identification recall without the registered patient name, before and after document-derived names, cue phrases and the name list.

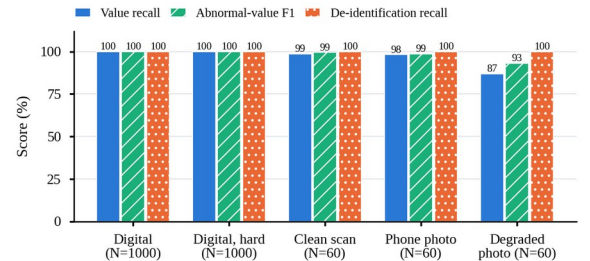


Fig. 8. Value recall, abnormal-value  $F_1$  and de-identification recall on digital text, scans and photographs.

### D. Performance and Cost

**Cryptographic cost.** Fig. 9 shows the median time to seal a file (new key, AES-256-GCM, key wrapping, fingerprint) and

to open it (unwrap, tag check, decryption, re-hash). Both scale linearly at about 150 MB/s: 6.7 ms and 7.1 ms for 1 MB, and under 70 ms for 10 MB. GCM adds no padding, so the only storage overhead is 384 bytes of metadata per file (0.04% at 1 MB).

**Concurrency.** Fig. 10 shows throughput and p95 latency for 1–100 concurrent clients on a data set of 20 patients with 50 records each, all shared with one doctor (no errors at any level). Batching state writes, keeping the audit trail in an append-only encrypted file, indexing consent and paging record lists cut the doctor’s list (a page of 25) from a p95 of 3.4 s to 68 ms at 50 clients (114 ms at 100) and uploads of 100 KB from 8.8 s to 204 ms (421 ms at 100). The record check, which decrypts and re-hashes a record, reached 1,620 requests/s at 50 clients with a p95 of 36 ms. Downloading a 5 MB file is CPU-bound at about 20 per second.

**Gas.** Fig. 11 lists the gas per contract operation. Registration first kept a per-patient array and three mappings (120.4k gas by relayer); it now stores one packed slot per record and lists records from the RecordAdded events, cutting it by 60% to 48.7k. The relayer can also anchor one Merkle root for many records, each still verifiable with its proof: 4.8k gas per record in batches of 10, 1.0k in batches of 50. Granting access costs 46.0k gas (48.7k by relayer), 38–41% less than the 78k of [17]; revoking costs 24.0k.

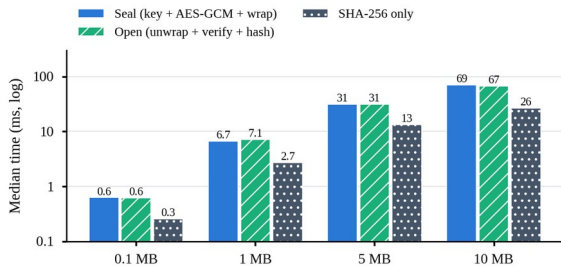


Fig. 9. Median time to seal and open a file, compared with hashing alone (log scale).

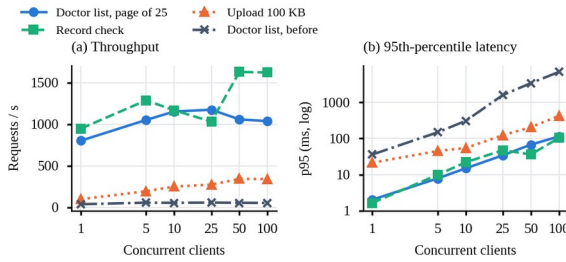


Fig. 10. Concurrency on 1,001 records: (a) throughput and (b) 95th-percentile latency for 1–100 clients; the grey line is the doctor’s list before paging.

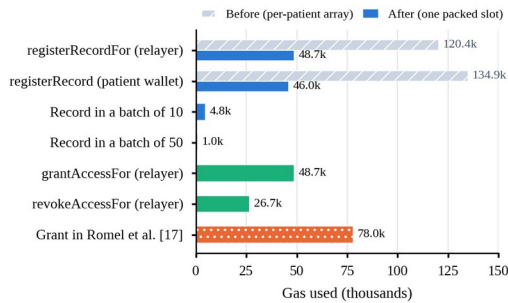


Fig. 11. Gas per contract operation before and after the storage change, per record in Merkle batches, and the grant cost in [17] (Hardhat).

## E. Usability and Localisation

Fig. 12 compares the automated plain-language scores before and after the rewrite. Before, 16 of 56 pages scored below 8 and the mean was 7.95; the public pages and the administrator pages were worst because they exposed ledger, hash and node terminology. Afterwards every page scored 9.7 or higher (mean 9.99), no page overflowed a 375-pixel phone screen and every button was at least 44 pixels tall. A second crawler opened each page in Nepali and in Hindi and found no interface text left in English; the only English strings remaining were names, identifiers and e-mail addresses.

## F. Comparison with Existing Systems

Table VIII sets the measured results beside the figures reported by recent systems. The testbeds differ, so the comparison is indicative rather than a benchmark. Where the same quantity is reported, MedLedger AI is comparable or cheaper: its grant transaction uses less gas than [17], and its per-file server time at 1 MB (about 20 ms, excluding the chain) is small next to the 0.7–1.4 s end-to-end time of [17], which includes the chain. The main difference is scope: none of the compared systems reports tamper-detection trials, a full access matrix or AI accuracy.

TABLE VIII  
QUANTITATIVE COMPARISON WITH REPORTED RESULTS

| System          | Reported evaluation                               | Key figures                                                                                                                                                            |
|-----------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HealthChain [9] | Simulation vs. baseline                           | ~30% faster access, ~50% fewer breaches                                                                                                                                |
| Singh [12]      | Gas vs. prior work                                | 16% lower gas                                                                                                                                                          |
| Katoon [14]     | Cross-chain sync                                  | < 195 ms                                                                                                                                                               |
| Romel [17]      | Gas, end-to-end latency                           | 78k gas / grant; 0.7–1.4 s for 1 MB                                                                                                                                    |
| MedLedger AI    | Gas, latency, load, tamper, access, AI, usability | 46.0k gas / grant, 48.7k / record (1.0k batched); 19.6 ms download (1 MB); 6,200 / 6,200 tamper cases caught; 81 / 81 access cells; abnormal-value F1 1.00 (synthetic) |

## G. Discussion and Threats to Validity

Per-file authenticated encryption, a single consent list and an asynchronous relayer combine without making users wait for the chain, and only fingerprints and pseudonymous addresses go on-chain. Binding keys to their record through associated data mattered: without it, a record swap passes the tag check. The server history is tamper-evident, not immutable; only entries confirmed on Ethereum inherit its immutability, and the relayer is trusted to submit, not to forge, decisions for patients without a wallet.

**Threats to validity.** *Construct:* the AI ground truth is synthetic, so perfect held-out scores show that the engine handles the generator’s variation, not real laboratories; the unseen-phrasing test (81.6%) is the more honest indicator. *Internal:* one team wrote the generator and the engine, and the 28 false alarms were resolved by correcting the ground truth; separate development seeds limit, but do not remove, this bias. *External:* all measurements used one machine and a local chain; Sepolia latency and fees are not yet measured. *Usability* was scored by crawlers, not people. A study under way addresses the first and last: 50–100 anonymised real reports labelled by hand (target  $\geq 95\%$  value recall on printed reports), 30 summaries rated by two doctors for accuracy, usefulness and harm, and 10–15 patients and 5 clinicians completing tasks in English, Nepali or Hindi, followed by the System Usability Scale.

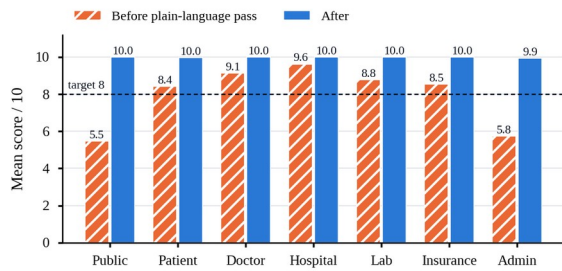


Fig. 12. Mean plain-language score per workspace before and after the rewrite (56 pages).

## VIII. CONCLUSION AND FUTURE WORK

MedLedger AI shows that encrypted storage, patient-controlled consent, a blockchain-anchored record history and privacy-preserving AI can be integrated in one usable platform. Every document is individually encrypted and verified on every access, every sharing decision is traceable on and off the chain, and reports are explained in plain language in three languages. The evaluation detected every one of 6,200 tampering attempts, confirmed the access policy for every role and action, removed every personal detail from 2,000 held-out synthetic reports even without the registered name, and found and classified every lab value, including SI units and laboratory-specific ranges. Future work will add HL7 FHIR interoperability, a mobile application, the real-report, clinician and user studies described above, evaluation of AI summaries with established metrics [19], and measurements on the Sepolia network under realistic load.

## REFERENCES

- [1] T.-T. Kuo, H.-E. Kim, and L. Ohno-Machado, "Blockchain distributed ledger technologies for biomedical and health care applications," *J. Amer. Med. Inform. Assoc.*, vol. 24, no. 6, pp. 1211–1220, 2017, doi: 10.1093/jamia/ocx068.
- [2] A. Hasselgren, K. Kralevska, D. Gligoroski, S. A. Pedersen, and A. Faxvaag, "Blockchain in healthcare and health sciences—A scoping review," *Int. J. Med. Inform.*, vol. 134, Art. no. 104040, 2020, doi: 10.1016/j.ijmedinf.2019.104040.
- [3] N. Ettaloui, S. Arezki, and T. Gadi, "Blockchain-based electronic health record: Systematic literature review," *Hum. Behav. Emerg. Technol.*, vol. 2024, no. 1, Art. no. 4734288, 2024, doi: 10.1155/hbe2/4734288.
- [4] A. Chandak, P. Chandak, and N. Soni, "Blockchain applications in electronic health records: A systematic review of qualitative and quantitative evidence," *BMC Med. Inform. Decis. Mak.*, vol. 26, no. 1, Art. no. 176, 2026, doi: 10.1186/s12911-026-03476-3.
- [5] D. Van Veen *et al.*, "Adapted large language models can outperform medical experts in clinical text summarization," *Nat. Med.*, vol. 30, no. 4, pp. 1134–1142, 2024, doi: 10.1038/s41591-024-02855-5.
- [6] K. Singhal *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, no. 7972, pp. 172–180, 2023, doi: 10.1038/s41586-023-06291-2.
- [7] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using blockchain for medical data access and permission management," in *Proc. 2nd Int. Conf. Open Big Data (OBD)*, 2016, pp. 25–30, doi: 10.1109/OBD.2016.11.
- [8] G. G. Dagher, J. Mohler, M. Milojkovic, and P. B. Marella, "Ancile: Privacy-preserving framework for access control and interoperability of electronic health records using blockchain technology," *Sustain. Cities Soc.*, vol. 39, pp. 283–297, 2018, doi: 10.1016/j.scs.2018.02.014.
- [9] G. Husnain *et al.*, "HealthChain: A blockchain-based framework for secure and interoperable electronic health records (EHRs)," *IET Commun.*, vol. 18, no. 19, pp. 1451–1473, 2024, doi: 10.1049/cmu2.12839.
- [10] A. Haddad, M. H. Habaebi, E. A. A. Elsheikh, M. R. Islam, S. A. Zabidi, and F. E. M. Suliman, "E2EE enhanced patient-centric blockchain-based system for EHR management," *PLoS ONE*, vol. 19, no. 4, Art. no. e0301371, 2024, doi: 10.1371/journal.pone.0301371.
- [11] N. U. A. Tahir *et al.*, "Blockchain-based healthcare records management framework: Enhancing security, privacy, and interoperability," *Technologies*, vol. 12, no. 9, Art. no. 168, 2024, doi: 10.3390/technologies12090168.
- [12] M. K. Singh, S. K. Pippal, and V. Sharma, "A blockchain-IPFS framework for secure, scalable, and interoperable healthcare data management," *SN Comput. Sci.*, vol. 6, no. 5, Art. no. 400, 2025, doi: 10.1007/s42979-025-03936-z.
- [13] J. Dutta and S. Barman, "Smart contract and blockchain-based secured approach for storing and sharing electronic health records," *Multimedia Tools Appl.*, vol. 84, no. 16, pp. 16883–16907, 2025, doi: 10.1007/s11042-024-19714-7.
- [14] P. M. Katon and A. V. Turukmane, "Interoperable blockchain network for healthcare data using Fabric, Ethereum and IPFS," *Discover Artif. Intell.*, vol. 5, Art. no. 308, 2025, doi: 10.1007/s44163-025-00564-7.
- [15] X. Pu, R. Jiang, Z. Song, Z. Liang, and L. Yang, "A medical big data access control model based on smart contracts and risk in the blockchain environment," *Front. Public Health*, vol. 12, Art. no. 1358184, 2024, doi: 10.3389/fpubh.2024.1358184.
- [16] A. Ullah, Z. Ullah, S. S. Rizvi, L. Gul, and S. J. Kwon, "Toward blockchain based electronic health record management with fine grained attribute based encryption and decentralized storage mechanisms," *Sci. Rep.*, vol. 15, no. 1, Art. no. 34542, 2025, doi: 10.1038/s41598-025-17875-5.
- [17] T. H. Romel, K. K. Paul, T. I. Ruhan, M. R. Mim, and A. S. M. L. Hoque, "A patient-centric blockchain framework for secure electronic health record management: Decoupling data storage from access control," 2025, *arXiv:2511.17464*.
- [18] M. Al-Garadi, T. Mungle, A. Ahmed, A. Sarker, Z. Miao, and M. E. Matheny, "Large language models in healthcare," 2025, *arXiv:2503.04748*.
- [19] E. Croxford *et al.*, "Evaluating clinical AI summaries with large language models as judges," *npj Digit. Med.*, vol. 8, Art. no. 640, 2025, doi: 10.1038/s41746-025-02005-2.
- [20] I. Neamatullah *et al.*, "Automated de-identification of free-text medical records," *BMC Med. Inform. Decis. Mak.*, vol. 8, Art. no. 32, 2008, doi: 10.1186/1472-6947-8-32.
- [21] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Anal. Recognit. (ICDAR)*, 2007, vol. 2, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.
- [22] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Yellow Paper, Shanghai version efc5f9a, Feb. 2025. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [23] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST, Gaithersburg, MD, USA, SP 800-38D, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.
- [24] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.
- [25] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845.
- [26] R. Bloemen, L. Logvinov, and J. Evans, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, no. 712, Sep. 2017. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-712>
- [27] W. Chang, G. Rocco, B. Millegan, N. Johnson, and O. Terbu, "ERC-4361: Sign-In with Ethereum," Ethereum Improvement Proposals, no. 4361, Oct. 2021. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-4361>